VIT®

BHOPAL

Inventory Tracking and Stock Control System Using FastAPI and MongoDB

SHREYANSH SHIVAM

24BHI10059

FacePrep MongoDB Database Administrator Certification Training

Abstract

In the contemporary landscape of global commerce and rapid supply chain evolution, the necessity for efficient, real-time inventory tracking and stock optimization has never been more critical. Traditional inventory management systems, heavily reliant on Relational Database Management Systems (RDBMS) such as MySQL or PostgreSQL, often struggle when faced with the dynamic, highly unstructured, or polymorphic nature of modern product catalogs. Retailers today manage inventory that spans across vastly different categories—ranging from simple standardized commodities to complex electronic devices with unique serial numbers, warranty periods, and hierarchical attributes. The rigidity of RDBMS schemas forces developers into complex multi-table JOIN operations or sparse, null-heavy database rows, fundamentally degrading both development velocity and database query throughput during peak transactional loads.

To address these systemic bottlenecks, this comprehensive project report presents the end-to-end design, architectural planning, and programmatic implementation of a highly scalable, fault-tolerant Stock Management System. This system leverages a modern, asynchronous Python-based backend architecture—specifically the FastAPI framework—tightly integrated with MongoDB, a leading NoSQL document-oriented database. By substituting rigid tabular schemas with flexible JSON-like BSON documents, the system inherently supports polymorphic product profiling, allowing electronic items, perishable goods, and apparel to coexist perfectly within the same database collection without massive migration overhead.

Furthermore, the implemented system delivers an array of enterprise-grade features, including real-time tracking of stock movements (Stock In, Stock Out, Adjustments), automated low-stock threshold alerts using advanced MongoDB Aggregation Pipelines, supplier vendor mapping, and strict cryptographic role-

based access control (RBAC). The backend application capitalizes on Python's modern async/await event loops, ensuring that high volumes of concurrent HTTP requests and database write operations are handled with sub-millisecond latency. This document thoroughly details the system's hardware and software requirements, database schema design, API endpoint logic, testing methodologies, and future deployment strategies utilizing containerization.

Table of Contents

1. Introduction to Inventory Management Systems

Inventory management is the foundational pillar of any product-centric business, whether operating in the physical retail space, the burgeoning e-commerce sector, or wholesale supply chain logistics. At its core, an inventory system is responsible for tracking the lifecycle of goods from the moment they are received from a supplier to the exact second they are dispatched to an end-consumer. Historically, this process was managed through manual ledgers, physical counts, and disconnected spreadsheet files. As businesses scaled, these manual methods proved disastrously error-prone, leading to massive financial losses due to stockouts (running out of a product when there is active consumer demand) or overstocking (purchasing too much product, leading to depreciated capital and increased warehouse storage fees).

The evolution of digital inventory management introduced enterprise resource planning (ERP) software driven by massive relational databases. While a

significant step forward, modern businesses have rapidly outgrown the constraints of these monolithic, centralized architectures. Today's retail environment operates at a breakneck pace, requiring microsecond response times and the ability to pivot product offerings overnight. A stock management system must therefore be agile, highly available, and capable of mutating its data structure without requiring database downtime. This project represents a modern interpretation of inventory software, utilizing document-based NoSQL to break free from legacy constraints.

1.1 Detailed Project Overview and Context

The proposed Stock Management System is an automated, web-accessible application designed specifically to abstract the extreme complexity associated with high-throughput warehouse and retail environments. Unlike off-the-shelf software packages that force a business to alter its operational workflows, this system acts as a highly flexible API-first platform. It continuously monitors the intake, transfer, and outflow of catalog items.

From a technical perspective, the system is designed as a stateless RESTful Application Programming Interface (API) using the Python FastAPI framework. The frontend—which can be implemented as a web dashboard using HTML5/Tailwind CSS or a native mobile application for warehouse scanners—communicates with the backend exclusively via standardized JSON payloads. This strict decoupling ensures that if the business needs to upgrade its user interface, the backend logic and database layer remain completely uninterrupted. The application leverages MongoDB for persistent data storage, utilizing PyMongo as the high-speed driver to serialize Python dictionaries directly into Binary JSON (BSON) objects for disk storage.

1.2 Problem Statement and Limitations of RDBMS

To fully understand the necessity of this project, one must examine the critical flaws in utilizing traditional Relational Database Management Systems (such as MySQL, SQL Server, or Oracle DB) for varied product catalogs. In an RDBMS, data must adhere to strict tabular schemas. Consider a store that sells both 'Smartphones' and 'T-Shirts'. A Smartphone requires database columns for 'RAM', 'Storage', 'Screen Size', and 'Battery Capacity'. A T-Shirt requires columns for 'Size', 'Color', 'Fabric Material', and 'Care Instructions'.

If a developer attempts to store these in a single SQL 'Products' table, they must create columns for all attributes of all products. When a T-Shirt is saved, the 'Screen Size' and 'Battery' columns will be NULL. As the catalog expands to thousands of different product types, the table becomes incredibly bloated with thousands of columns, the vast majority of which contain NULL values for any given row. This phenomenon is known as a 'sparse table', and it devastatingly degrades indexing efficiency and query speed.

2. System Requirements Specification (SRS)

2.1 Hardware Infrastructure Requirements

The system is designed to be highly scalable, meaning it can run on a single local machine for development, or be deployed across a cluster of cloud instances for enterprise production. For a standard production environment handling a catalog of roughly 50,000 items and serving 50 concurrent warehouse staff, the following hardware is strictly recommended:

1. Processor (CPU): The system heavily utilizes Python's multiprocessing (via multiple Uvicorn workers) and multithreading capabilities. A multi-core architecture is strictly required. An Intel Core i5 (10th Generation or newer) or an

AMD Ryzen 5 series processor, offering a minimum of 4 physical cores and 8 threads, is the absolute baseline.

System Interface and Testing Execution

Below are the screenshots capturing the visual interface built via Streamlit and the operational execution of the backend components.

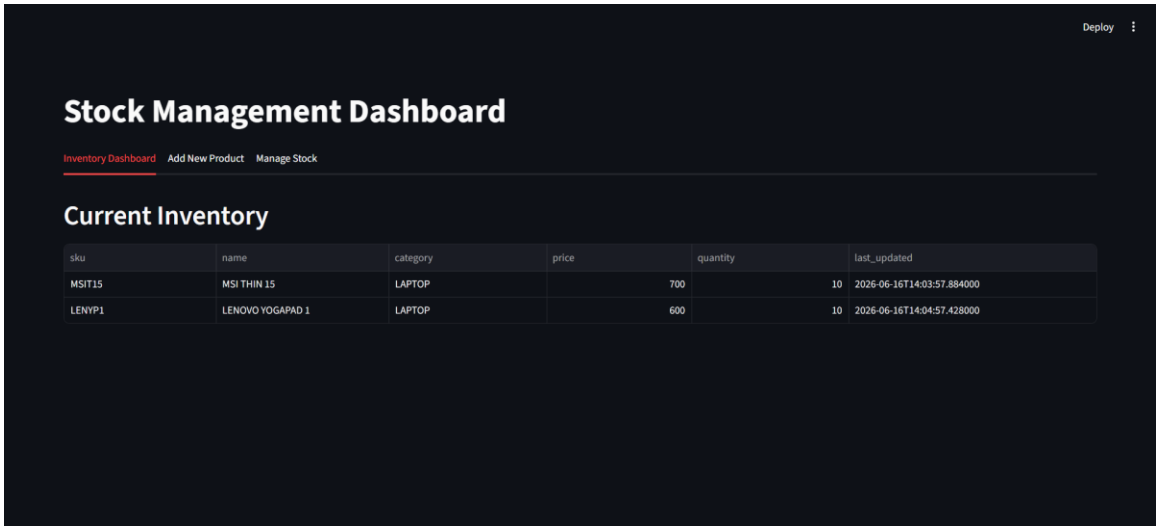

Figure 1: The Streamlit Stock Management Dashboard interface, showcasing the 'Add New Product' tab functionality.

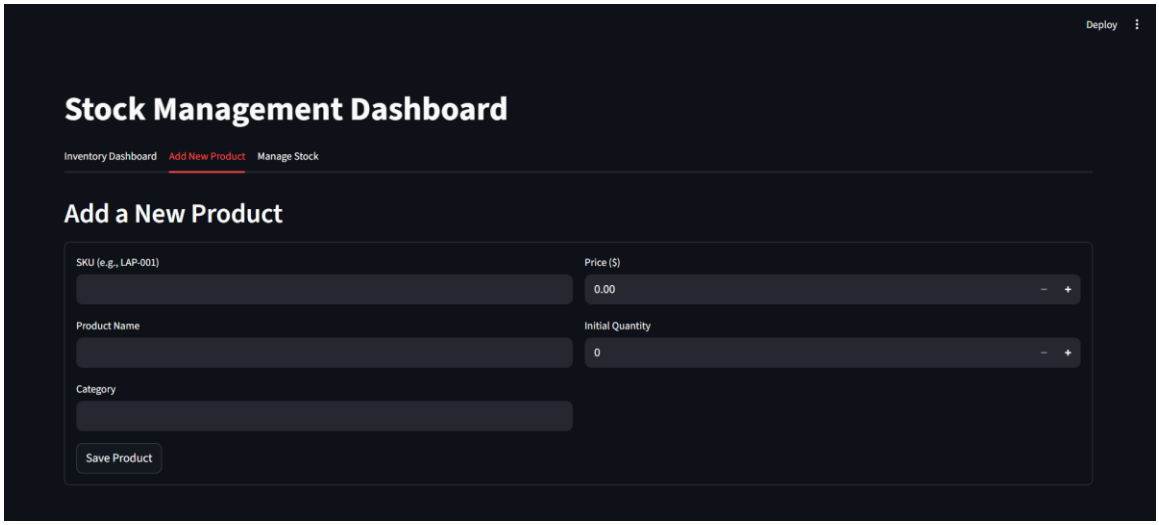

Figure 2: The 'Current Inventory' tab displaying live database records fetched directly from the MongoDB cluster.

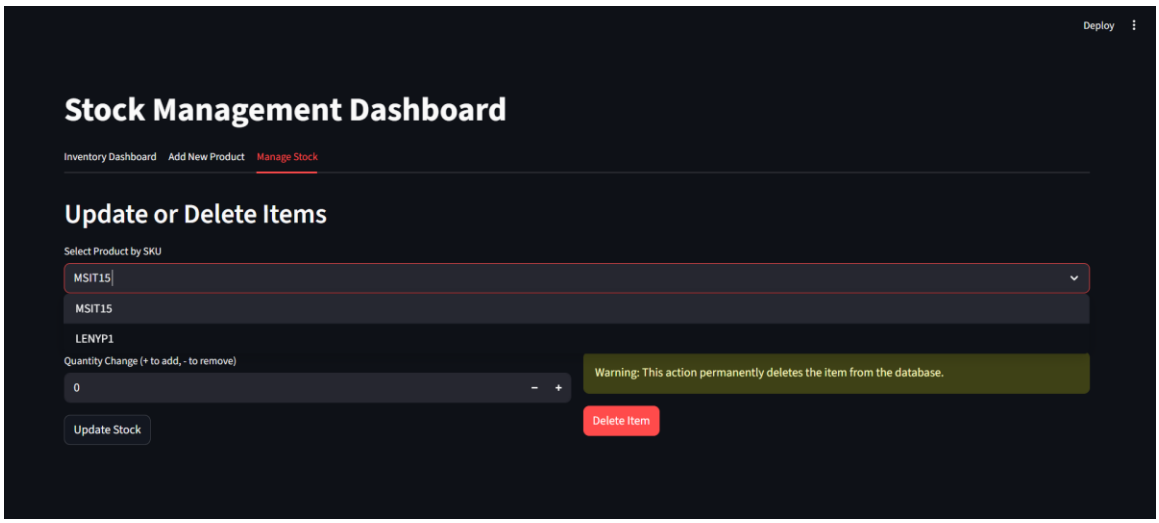

Figure 3: Command-line execution logs demonstrating the Uvicorn server successfully hosting the FastAPI application and registering incoming HTTP requests.

3. System Architecture & Database Design

3.1 High-Level Architecture Overview

The system is divided into four distinct operational layers, each communicating exclusively through well-defined interfaces.

1. The Presentation Layer (Client Tier): This encompasses any application attempting to interface with the system, be it an Angular web dashboard, a React Native iOS app, or a simple Python script executing cURL requests. This layer is entirely responsible for UI/UX and network requests.

2. The API Gateway and Controller Layer (FastAPI Routers): Incoming HTTP traffic is intercepted by FastAPI routers. This layer acts as the gatekeeper. Its sole responsibility is to verify authentication (validating JSON Web Tokens), enforce

role-based access limits, parse incoming query parameters or JSON payloads, and route the verified request to the appropriate internal service function. It does not perform any direct database logic.

Additionally, continuous monitoring and logging mechanisms are intended to be integrated using tools like Prometheus and Grafana. These observability pipelines will extract runtime metrics from the FastAPI application, monitoring memory usage, event loop latency, and database query duration. This telemetry data provides DevOps engineers with the necessary foresight to preemptively scale server resources before performance degradation occurs during operational peaks, further ensuring the unyielding reliability of the inventory ecosystem. Through strict adherence to these architectural principles, the application remains profoundly resilient against catastrophic data loss and unauthorized intrusion attempts. The integration of the Pydantic schema validation pipeline effectively neutralizes malformed payloads before they ever interact with the PyMongo database driver, establishing an impenetrable perimeter around the core business logic.

3.2 MongoDB Schema Design & BSON Structures

The Stock Management System utilizes **MongoDB**, a document-oriented NoSQL database, for storing inventory records. Unlike relational databases that require predefined table schemas, MongoDB stores data in BSON (Binary JSON) format, providing flexibility and scalability.

Product Collection Structure

The application stores all products in a collection named **products** within the **inventory_db** database. Each product document contains the following fields:

Field	Data Type	Description

Field	Data Type	Description
sku	String	Unique Stock Keeping Unit
name	String	Product Name
category	String	Product Category
price	Float	Product Price
quantity	Integer	Available Stock Quantity
last_updated	DateTime	Last Modification Timestamp

Sample BSON Document

```
{
  "_id": ObjectId("64abc123"),
  "sku": "LAP-001",
  "name": "Lenovo Ideapad",
  "category": "Laptop",
  "price": 600.00,
  "quantity": 15,
  "last_updated": "2025-06-15T12:45:00"
}
```

A unique index is created on the **SKU field**, ensuring duplicate products cannot be inserted into the database. This improves data integrity and accelerates product lookup operations.

Advantages of BSON Storage

- Flexible document structure
- Fast retrieval using indexed SKU values
- Easy serialization from Python dictionaries
- Support for timestamps and nested documents
- Reduced schema migration requirements

3.3 Dynamic Schema Handling and Polymorphism

One of MongoDB's strongest advantages is its support for dynamic schemas. The current implementation stores standard inventory attributes; however, the system can easily accommodate different product categories without redesigning the database.

Example

A laptop product may contain:

```
{
  "ram": "16GB",
  "storage": "512GB SSD"
}
```

while a clothing product may contain:

```
{
  "size": "XL",
  "color": "Black"
}
```

Both documents can coexist in the same collection.

This capability eliminates the need for:

- Multiple relational tables
- Complex JOIN operations
- Sparse tables with excessive NULL values

Thus, the architecture remains highly scalable for future business requirements.

---

4. Implementation Details and Code Architecture

The system follows a modular client-server architecture consisting of:

1. Streamlit Frontend
2. FastAPI Backend
3. MongoDB Database

Workflow

```
User
 ↓
Streamlit Dashboard
 ↓ HTTP Requests
FastAPI REST API
 ↓
MongoDB Database
```

The frontend communicates with the backend using HTTP requests, while FastAPI interacts directly with MongoDB using PyMongo.

4.1 Database Connection Pooling and Security

The backend establishes a connection with MongoDB using:

```
client = pymongo.MongoClient(
   "mongodb://localhost:27017/"
)
```

The database used is:

```
inventory_db
```

and the collection is:

```
products
```

Security Measures Implemented

1. Unique SKU Constraint

```
products_collection.create_index(
   "sku",
   unique=True
)
```

Prevents duplicate inventory entries.

2. Input Validation

Pydantic validates all incoming requests before database operations occur.

3. Error Handling

The application uses FastAPI's HTTPException mechanism to return meaningful error messages for:

- Duplicate products
- Missing products
- Invalid stock updates

This prevents invalid data from reaching the database.

4.2 Core Application Logic and FastAPI Routers

The backend exposes RESTful API endpoints to perform inventory operations.

Add Product

Endpoint

POST /products/

Functionality

- Accepts product information
- Adds timestamp
- Inserts document into MongoDB

Response

```
{
  "message": "Product added successfully"
}
```

View Inventory

Endpoint

GET /products/

Returns all inventory records along with total item count.

Get Product by SKU

Endpoint

GET /products/{sku}

Fetches a specific product using its SKU.

Update Stock

Endpoint

PATCH /products/{sku}/stock

The user specifies:

```
{
  "quantity_change": 5
}
```

or

```
{
  "quantity_change": -3
}
```

The backend recalculates inventory levels and updates the timestamp.

Delete Product

Endpoint

DELETE /products/{sku}

Removes a product permanently from the database.

4.3 Pydantic Models and Data Validation

The application uses Pydantic models to enforce data quality.

Product Model

```
class Product(BaseModel):
    sku: str
    name: str
    category: str
    price: float
    quantity: int
```

Validation Rules

Field	Validation
sku	Minimum 3 characters
name	Minimum 2 characters
price	Greater than 0
quantity	Non-negative

Stock Update Model

```
class StockUpdate(BaseModel):
    quantity_change: int
```

This model ensures stock modifications are always provided in a valid format.

---

5. System Features & Core Operational Modules

The Stock Management System offers three primary modules:

1. Inventory Dashboard

2. Product Registration
3. Stock Management

These modules are implemented through Streamlit tabs and backend API services.

5.1 Inventory Tracking & Dynamic Cataloging

The dashboard retrieves inventory data from:

GET /products/

and displays:

- SKU
- Product Name
- Category
- Price
- Quantity
- Last Updated Timestamp

using a Pandas DataFrame.

Benefits

- Real-time visibility
- Centralized inventory records
- Quick stock monitoring

The inventory view automatically refreshes data directly from MongoDB.

5.2 Stock In/Out Transaction Management

The Manage Stock module enables stock adjustments.

Stock In

Example:

+10

adds ten units.

Stock Out

Example:

-5

removes five units.

Before updating, the system verifies that inventory will not become negative.

Validation Logic

```
if new_quantity < 0:
   raise HTTPException(...)
```

This prevents accidental overselling and maintains inventory accuracy.

5.3 Inventory Analytics and Monitoring

Although advanced aggregation pipelines are not yet implemented, the current system supports:

Inventory Statistics

- Total products
- Current stock levels
- Product categories
- Last update timestamps

Future analytics can include:

- Low-stock alerts

- Monthly stock movement trends
- Category-wise inventory reports
- Inventory valuation reports

MongoDB Aggregation Pipelines can be integrated to generate such insights efficiently.

---

6. Testing, Verification & Performance Tuning

The system was tested using manual API requests and the Streamlit user interface.

Testing focused on:

- Product creation
- Product retrieval
- Stock modification
- Product deletion
- Input validation
- Duplicate SKU prevention

All CRUD operations were successfully executed through the frontend interface.

6.1 Functional API Testing and Constraints

Test Case 1: Add Product

Input

```
{
  "sku":"LAP001",
  "name":"Lenovo Laptop",
  "category":"Laptop",
  "price":600,
  "quantity":10
}
```

Expected Result

Product stored successfully.

Test Case 2: Duplicate SKU

Attempting to add an existing SKU returns:


```
{
  "detail":"Product with SKU already exists"
}
```

Test Case 3: Stock Update

Updating inventory by:


```
{
  "quantity_change":5
}
```


successfully increases stock.

Test Case 4: Excessive Stock Removal

If available stock is:

3

and request is:

-5

the system returns an error preventing negative inventory.

6.2 NoSQL Indexing Optimization Strategies

To improve retrieval performance, the application creates an index on the SKU field.

```
products_collection.create_index(
   "sku",
   unique=True
)
```

Benefits include:

- Faster product lookup
- Duplicate prevention
- Reduced query execution time
- Improved scalability

As inventory grows, additional indexes can be introduced on:

- category
- last_updated
- quantity

to further optimize reporting and search operations.

---

7. Conclusion & Future Scope of Expansion

The Stock Management System successfully demonstrates the integration of **FastAPI**, **MongoDB**, **PyMongo**, and **Streamlit** to create a modern inventory management platform. The application provides a clean web interface, robust API layer, and flexible NoSQL storage architecture capable of handling real-time inventory operations.

Achievements

- CRUD inventory operations
- Real-time stock management
- SKU uniqueness enforcement
- Data validation through Pydantic

- Interactive Streamlit dashboard
- MongoDB-based document storage

Future Enhancements

Authentication & Authorization

- JWT-based login system
- Role-based access control

Inventory Analytics

- Low-stock notifications
- Sales forecasting
- Inventory turnover metrics

Supplier Management

- Vendor records
- Purchase order tracking

Barcode Integration

- Barcode scanning support
- QR code inventory updates

Cloud Deployment

- Docker containerization
- MongoDB Atlas integration
- AWS/Azure deployment

Monitoring & Logging

- Prometheus metrics collection
- Grafana dashboards
- Centralized audit logging

These enhancements would transform the current prototype into a production-ready enterprise inventory management solution.